Speech recognition

In the previous sections, we saw how RNNs can be used to learn patterns of many different time sequences. In this section, we will look at how these models can be used for the problem of recognizing and understanding speech. We will give a brief overview of the speech recognition pipeline and provide a high-level view of how we can use neural networks in each part of the pipeline. In order to know more about the methods discussed in this section, we would like you to refer to the references.

Speech recognition pipeline

Speech recognition tries to find a transcription of the most probable word sequence considering the acoustic observations provided; this is represented by the following:

transcription = argmax(P(words | audio features))

This probability function is typically modeled in different parts (note that the normalizing term P (audio features) is usually ignored):

*P (words | audio features) = P (audio features | words) * P (words)*

*= P (audio features | phonemes) * P (phonemes | words) * P (words)*

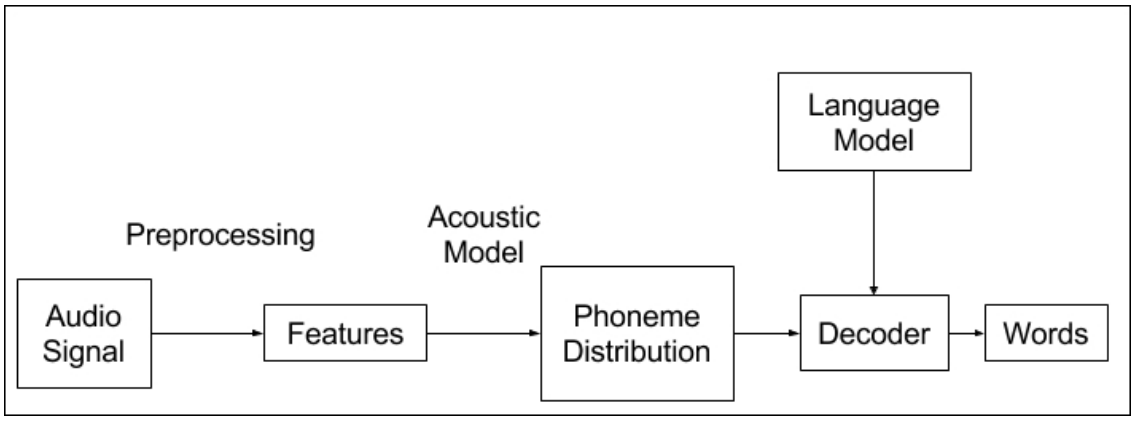
NOTE

What are phonemes?

Phonemes are a basic unit of sound that define the pronunciation of words. For example, the word "bat" is composed of three phonemes `/b/`, `/ae/`, and `/t/`. Each phoneme is

tied to a specific sound. Spoken English consists of around 44 phonemes.

Each of these probability functions will be modeled by different parts of the recognition system. A typical speech recognition pipeline takes in an audio signal and performs preprocessing and feature extraction. The features are then used in an acoustic model that tries to learn how to distinguish between different sounds and phonemes: *P (audio features | phonemes)*. These phonemes are then matched to characters or words with the help of pronunciation dictionaries: *P(phonemes | words)*. The probabilities of the words extracted from the audio signal are then combined with the probabilities of a language model, *P(words)*. The most likely sequence is then found via a decoding search step that searches for the most likely sequence(see *Decoding* section). A high-level overview of this speech recognition pipeline is described in the following figure:



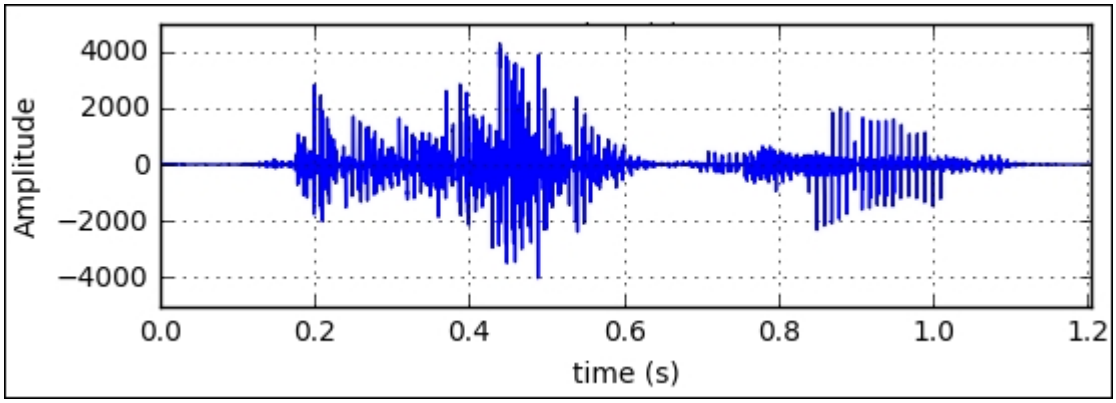
Overview of a typical speech recognition pipeline

Large, real-world vocabulary speech recognition pipelines are based on this same pipeline; however, they use a lot of tricks and heuristics in each step to make the problem tractable. While these details are out of the scope of this section, there is open source software available—Kaldi [29]—that allows you to train a speech recognition system with advanced pipelines.

In the next sections, we will briefly describe each of the steps in this standard pipeline and how deep learning can help improve these steps.

Speech as input data

Speech is a type of sound that typically conveys information. It is a vibration that propagates through a medium, such as air. If these vibrations are between 20 Hz and 20 kHz, they are audible to humans. These vibrations can be captured and converted into a digital signal so that they can be used in audio signal processing on computers. They are typically captured by a microphone after which the continuous signal is sampled at discrete samples. A typical sample rate is 44.1 kHz, which means that the amplitude of the incoming audio signal is measured 44,100 times per second. Note that this is around twice the maximum human hearing frequency. A sampled recording of someone saying "hello world" is plotted in the following figure:



Speech signal of someone saying "hello world" in the time domain

Preprocessing

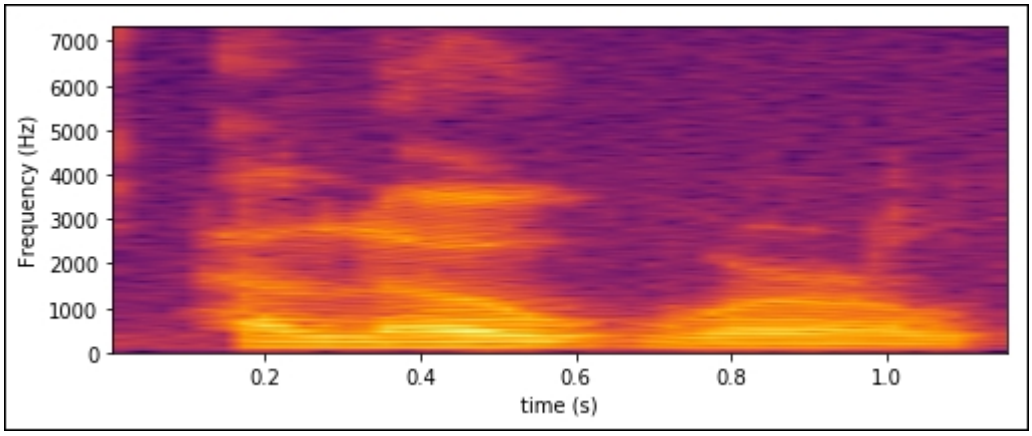
The recording of the audio signal in the preceding figure is recorded over 1.2 seconds. To digitize the audio, it is sampled 44,100 times per second (44.1 kHz). This means that roughly 50,000 amplitude samples were taken for this 1.2-second audio signal.

For only a small example, these are a lot of points over the time dimension. To reduce the size of the input data, these audio signals are typically preprocessed to reduce the number of time steps before feeding them into speech recognition algorithms. A typical transformation transforms a signal to a spectrogram, which is a representation of how the frequencies in the signal change over time, see the next figure.

This spectral transformation is done by dividing the time signal in overlapping windows and taking the Fourier transform of each of these win-

dows. The Fourier transform decomposes a signal over time into frequencies that make up the signal [30]. The resulting frequencies responses are compressed into fixed frequency bins. This array of frequency bins is also known as a filter banks. A filter bank is a collection of filters that separate out the signal in multiple frequency bands.

Say the previous "hello world" recording is divided into overlapping windows of 25 ms with a stride of 10 ms. The resulting windows are then transformed into a frequency space with the help of a windowed Fourier transform. This means that the amplitude information for each time step is transformed into amplitude information for each frequency. The final frequencies are mapped to 40 frequency bins according to a logarithmic scale, also known as the Mel scale. The resulting filter bank spectrogram is shown in the following figure . This transformation resulted in reducing the time dimension from 50,000 to 118 samples, where each sample is a vector of size 40.



Mel spectrum of speech signal from the previous figure

Especially in older speech recognition systems, these Mel-scale filter banks are even more processed by decorrelation to remove linear dependencies. Typically, this is done by taking a **discrete cosine transform** (**DCT**) of the logarithm of the filter banks. This DCT is a variant of the Fourier transform. This signal transformation is also known as **Mel Frequency Cepstral Coefficients** (**MFCC**).

More recently, deep learning methods, such as convolutional neural networks, have learned some of these preprocessing steps [31], [32].

Acoustic model

In speech recognition, we want to output the words being spoken as text. This can be done by learning a time-dependent model that takes in a sequence of audio features, as described in the previous section, and outputs a sequential distribution of possible words being spoken. This model is called the acoustic model.

The acoustic model tries to model the likelihood that a sequence of audio features was generated by a sequence of words or phonemes: *P (audio features | words) = P (audio features | phonemes) * P (phonemes | words)*.

A typical speech recognition acoustic model, before deep learning became popular, would use **hidden Markov models (HMMs)** to model the temporal variability of speech signals [33], [34]. Each HMM state emits a mixture of Gaussians to model the spectral features of the audio signal. The emitted Gaussians form a **Gaussian mixture model (GMM)**, and they determine how well each HMM state fits in a short window of acoustic features. HMMs are used to model the sequential structure of data, while GMMs model the local structure of the signal.

The HMM assumes that successive frames are independent given the hidden state of the HMM. Because of this strong conditional independence assumption, the acoustic features are typically decorrelated.

Deep belief networks

The first step in using deep learning in speech recognition is to replace GMMs with **deep neural networks (DNN)** [35]. DNNs take a window of feature vectors as input and output the posterior probabilities of the HMM states: *P (HMM state | audio features)*.

The networks used in this step are typically pretrained as a general model on a window of spectral features. Usually, **deep belief networks (DBN)** are used to pretrain these networks. The generative pretraining creates many layers of feature detectors of increased complexity. Once generative pretraining is finished, the network is discriminatively fine-tuned to classify the correct HMM states, based on the spectral features. HMMs in these hybrid models are used to align the segment classifications provided by the DNNs to a temporal classification of the full label sequence.

These DNN-HMM models have been shown to achieve better phone recognition than GMM-HMM models [36].

Recurrent neural networks

This section describes how RNNs can be used to model sequential data. The problem with the straightforward application of RNNs on speech recognition is that the labels of the training data need to be perfectly aligned with the input. If the data isn't aligned well, then the input to output mapping will contain too much of noise for the network to learn anything. Some early attempts tried to model the sequential context of the acoustic features by using hybrid RNN-HMM models, where the RNNs would model the emission probabilities of the HMM models, much in the same way that DBNs are used [37].

Later experiments tried to train LSTMs (see section on *Long Short Term Memory*) to output the posterior probability of the phonemes at a given frame [38].

The next step in speech recognition would be to get rid of the necessity of having aligned labeled data and removing the need for hybrid HMM models.

CTC

Standard RNN objective functions are defined independently for each sequence step, each step outputs its own independent label classification. This means that training data must be perfectly aligned with the target labels. However, a global objective function that maximizes the probability of a full correct labeling can be devised. The idea is to interpret the network outputs as a conditional probability distribution over all possible labeling sequences, given the full input sequence. The network can then be used as a classifier by searching for the most probable labeling given the input sequence.

Connectionist Temporal Classification (**CTC**) is an objective function that defines a distribution over all the alignments with all the output sequences [39]. It tries to optimize the overall edit distance between the out-

put sequence and the target sequence. This edit distance is the minimum number of insertions, substitutions, and deletions required to change the output labeling to target labeling.

A CTC network has a softmax output layer for each step. This softmax function outputs label distributions for each possible label plus an extra blank symbol (Ø). This extra blank symbol represents that there is no relevant label at that time step. The CTC network will thus output label predictions at any point in the input sequence. The output is then translated into a sequence labeling by removing all the blanks and repeated labels from the paths. This corresponds to outputting a new label when the network switches from predicting no label to predicting a label or from predicting one label to another. For example, "ØaaØabØØ" gets translated into "aab". This has as effect that only the overall sequence of labels has to be correct, thus removing the need for aligned data.

Doing this reduction means that multiple output sequences can be reduced to the same output labeling. To find the most likely output labeling, we have to add all the paths that correspond to that labeling. The task of searching for this most probable output labeling is known as decoding (see the *Decoding* section).

An example of such a labeling in speech recognition could be outputting a sequence of phonemes, given a sequence of acoustic features. The CTC objective's function, built on top of an LSTM, has been to give state-of-the-art results on acoustic modeling and to remove the need of using HMMs to model temporal variability [40], [41].

Attention-based models

An alternative to using the CTC sequence to sequence a model is an attention-based model [42]. These attention models have the ability to dynamically pay attention to parts of the input sequence. This allows them to automatically search for relevant parts of the input signal to predict the right phoneme, without having to have an explicit segmentation of the parts.

These attention-based sequence models are made up of an RNN that decodes a representation of the input into a sequence of labels, which are phonemes in this case. In practice, the input representation will be generated by a model that encodes the input sequence into a suitable representation. The first network is called the decoder network, while the latter is called the encoder network [43].

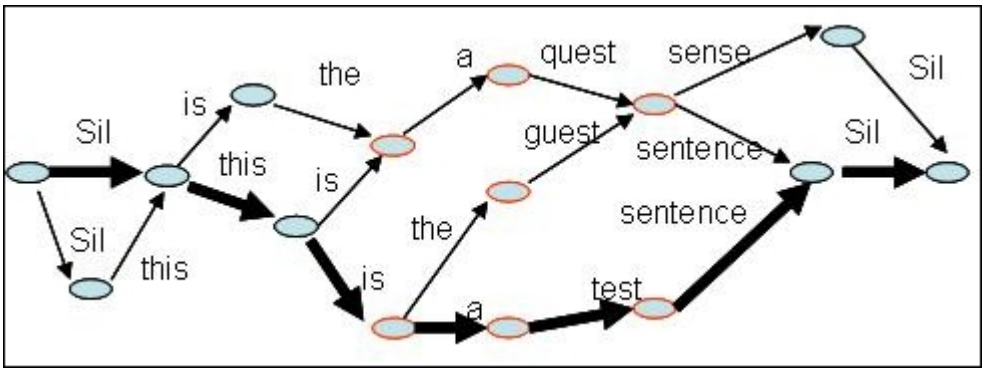
The decoder is guided by an attention model that focuses each step of the decoder on an attention window over encoded input. The attention model can be driven by a combination of context (what it is focusing on) or location-based information (where it is focusing on). The decoder can then use the previous information and the information from the attention window to output the next label (phoneme).

Decoding

Once we model the phoneme distribution with the acoustic model and train a language model (see the *Language Modelling* section), we can combine them together with a pronunciation dictionary to get a probability function of words over audio features:

*P (words | audio features) = P (audio features | phonemes) * P (phonemes | words) * P (words)*

This probability function doesn't give us the final transcript yet; we still need to perform a search over the distribution of the word sequence to find the most likely transcription. This search process is called decoding. All possible paths of decoding can be illustrated in a lattice data structure:



A pruned word lattice [44]

The most likely word sequence, given a sequence of audio features, is found by searching through all the possible word sequences [33]. A popular search algorithm based on dynamic programming that guarantees it could find the most likely sequence is the Viterbi algorithm [45]. This algorithm is a breadth-first search algorithm that is mostly associated with finding the most likely sequence of states in an HMM.

For large vocabulary speech recognition, the Viterbi algorithm becomes intractable for practical use. So in practice, heuristic search algorithms, such as beam search, are used to try and find the most likely sequence. The beam search heuristic only keeps the n-best solutions during the search and assumes that all the rest don't lead to the most likely sequence.

Many different decoding algorithms exist [46] and the problem of finding the best transcription from the probability function is mostly seen as unsolved.

End-to-end models

We want to conclude this chapter by mentioning end-to-end techniques. Deep learning methods, such as CTC [47], [48] and attention based models [49], have allowed us to learn the full speech recognition pipeline in an end-to-end fashion. They do so without modeling phonemes explicitly. This means that these end-to-end models will learn acoustic and language models in one single model and directly output a distribution over words. These models illustrate the power of deep learning by combining everything in one model; with this, the model becomes conceptually easier to understand. We speculate that this will lead to speech recognition being recognized as a solved problem in the next few years.